



SENTENCIA SIGNAL

Al trabajar con triggers, funciones y procedimientos almacenados, la gestión de excepciones se vuelve importante para evitar la finalización abrupta de los procedimientos y proporcionar información de error significativa.

La sentencia SIGNAL es la forma de devolver un error. Proporciona información del error a un controlador, a una parte externa de la aplicación o al cliente. Además, permite tener el control sobre las características del error como el número de error, el valor de SQLSTATE, un mensaje con la descripción del error, etc. De esta manera se puede gestionar eficazmente diferentes tipos de excepciones en MySQL y controlar con precisión el manejo de errores.

Descripción general de la sentencia SIGNAL

```
SIGNAL SQLSTATE [VALUE] sqlstate_value  
    SET MESSAGE_TEXT = simple_value,  
    SET MYSQL_ERRNO = simple_value  
    [, ...] ;
```

SQLSTATE [VALUE] representa un valor SQLSTATE (no nulo) que se define como una cadena literal de 5 caracteres. Los dos primeros caracteres de este valor indica su clase de error según los siguientes criterios:

- **Clase '00': éxito.** No válido. Los valores SQLSTATE que comienzan con '00' indican éxito y no son válidos para SIGNAL.
- **Clase = '01': advertencia.** Representa una condición de advertencia definida por el usuario no controlada (`MYSQL_ERRNO = ER_SIGNAL_WARN`). El valor de la variable de sistema "warning_count" aumenta y con la sentencia `SHOW WARNINGS` muestra la señal.
- **Clase = '02': no encontrado.** Representa una condición de no encontrado definida por el usuario no controlada (`MYSQL_ERRNO = ER_SIGNAL_NOT_FOUND`). Los controladores "NOT FOUND" captan la señal. No tiene efecto en los cursores. Si la señal no se controla en una función almacenada, las sentencias finalizan.
- **Clase > '02': excepción.** Representa una condición de excepción definida por el usuario no controlada (`MYSQL_ERRNO = ER_SIGNAL_EXCEPTION`). Los controladores `SQLEXCEPTION` detectan la señal. Si la señal no se



gestiona en una función almacenada, las sentencias finalizan.

Para señalar un valor SQLSTATE genérico, utilice '45000', que significa "excepción definida por el usuario no controlada".

También, se puede declarar una variable que contenga el valor SQLSTATE. De esta forma, se puede reemplazar la declaración de este valor en la sentencia SIGNAL.

```
DECLARE divide_by_zero CONDITION FOR SQLSTATE '22012';

IF divisor = 0 THEN
    SIGNAL divide_by_zero;
END IF;
```

Visualización de errores

Los valores de error accesibles tras la ejecución de SIGNAL son el valor SQLSTATE generado por la sentencia SIGNAL y los elementos MESSAGE_TEXT y MYSQL_ERRNO. Estos valores están disponibles mediante los siguientes métodos:

- mysql_sqlstate() devuelve el valor SQLSTATE.
- mysql_errno() devuelve el valor MYSQL_ERRNO.
- mysql_error() devuelve el valor MESSAGE_TEXT.

A nivel de SQL, la salida de SHOW WARNINGS y SHOW ERRORS indica los valores MYSQL_ERRNO y MESSAGE_TEXT en las columnas Código y Mensaje.

Ejemplo 1

En este ejemplo, creamos un procedimiento que acepta la forma corta de los títulos de grado y devuelve sus formas completas. Si proporcionamos un título no válido se genera un mensaje de error utilizando la declaración SIGNAL.



```
DELIMITER //
CREATE PROCEDURE example(IN degree VARCHAR(20), OUT full_form
Varchar(50))
BEGIN
    IF degree='B-Tech' THEN
        SET full_form = 'Bachelor of Technology';
    ELSEIF degree='M-Tech' THEN
        SET full_form = 'Master of Technology';
    ELSEIF degree='BSC' THEN
        SET full_form = 'Bachelor of Science';
    ELSEIF degree='MSC' THEN
        SET full_form = 'Master of Science';
    ELSE
        SIGNAL SQLSTATE '01000'
        SET MESSAGE_TEXT = 'Choose from the existing values',
MYSQL_ERRNO = 12121;
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Given degree is not valid',
MYSQL_ERRNO = 1001;
    END IF;
END //

DELIMITER ;
```

Podemos llamar al procedimiento anterior para recuperar el resultado o, en caso de pasar un valor inválido, obtener un mensaje de error.

```
CALL example('BSC', @fullform);

SELECT @fullform; -- @fullform//Licenciatura en Ciencia

CALL example ('BBC', @fullform); -- ERROR 1001 (45000): Given
degree is not valid
```

Ejemplo 2

A continuación se muestra otro ejemplo que demuestra el manejo de excepciones con la declaración SIGNAL. Aquí, declaramos una condición y usamos SIGNAL para activar excepciones basadas en ciertas condiciones.



```
DELIMITER //  
CREATE PROCEDURE example (num INT)  
BEGIN  
    DECLARE testCondition CONDITION FOR SQLSTATE '45000';  
    IF num < 0 THEN  
        SIGNAL SQLSTATE '01000';  
    ELSEIF num > 150 THEN  
        SIGNAL SQLSTATE '45000';  
    END IF;  
END //  
DELIMITER ;
```

Podemos llamar al procedimiento anterior para recuperar el resultado o, en caso de pasar un valor inválido, obtener un mensaje de error.

```
CALL example(15); -- Query OK, 0 rows affected (0.00 sec)  
  
CALL example(160); -- ERROR 1644 (45000): Unhandled  
user-defined exception condition
```

Ejemplo 3

Puede personalizar los mensajes de error usando SET MESSAGE_TEXT con la declaración SIGNAL.



```
DELIMITER //
```

```
CREATE PROCEDURE example (num INT)
```

```
BEGIN
```

```
  DECLARE testCondition CONDITION FOR SQLSTATE '45000';
```

```
  IF num < 0 THEN
```

```
    SIGNAL SQLSTATE '01000';
```

```
  ELSEIF num > 150 THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Number higher than
```

```
the limit set';
```

```
  END IF;
```

```
END //
```

```
DELIMITER ;
```



```
CALL example(20); -- Query OK, 0 rows affected (0.00 sec)
```



```
CALL example(160); -- ERROR 1644 (45000): Number higher than
```

```
the limit set
```

Uso de Handlers en Procedimientos Almacenados

Los handlers en MySQL son herramientas fundamentales para manejar errores o condiciones especiales que pueden surgir durante la ejecución de procedimientos almacenados. Entender su funcionamiento es clave para garantizar la robustez de nuestras rutinas.

Un handler es una instrucción que define cómo responder ante ciertas condiciones (errores, advertencias o situaciones específicas) que pueden ocurrir dentro de un bloque de código SQL. Los handlers pueden manejar diferentes tipos de errores:

- Errores específicos: mediante un código de error de MySQL o un valor SQLSTATE.
- Condiciones generales: como SQLEXCEPTION (errores en general), SQLWARNING (advertencias), o NOT FOUND (datos no encontrados).

También, podemos definir 2 tipos de handler:

- EXIT: finaliza la ejecución del bloque actual cuando se dispara.
- CONTINUE: permite que el código siga ejecutándose luego de manejar la condición.



Los handlers solo se aplican dentro del bloque BEGIN ... END en el que fueron declarados y a partir de la línea en que aparecen. A continuación mostramos un ejemplo sobre cómo aplicar un handler dentro de un bloque SQL.

```
BEGIN
  DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    SELECT 'Error capturado';
  -- Este handler aplicará sólo desde aquí hacia abajo
  DELETE FROM tabla_inexistente;
END;
```

Si un error ocurre fuera del alcance del handler (por ejemplo, antes de su declaración o en otro bloque), el handler no se activa.

También se pueden declarar varios handlers dentro del mismo bloque SQL. Se pueden anidar por medio de sub bloques, haciendo que los handlers más cercanos (locales) tengan prioridad y se ejecuten primero, siguiendo por aquellos más generales (globales). En caso de que se encuentren varios handlers dentro del mismo nivel, hay que tener en cuenta el orden en que MySQL trata estos casos:

- 1) Handler por código de error de MySQL.
- 2) Handler por valor SQLSTATE.
- 3) Handler general: NOT FOUND, SQLWARNING, SQLEXCEPTION.

Ejemplo 4

Supongamos que tenemos un procedimiento que intenta eliminar una tabla. Si esta tabla no existe, MySQL genera un error con código SQLSTATE '42502'.

```
CREATE PROCEDURE borrar_tabla()
BEGIN
  DECLARE CONTINUE HANDLER FOR SQLSTATE '42502'
    SELECT 'La tabla no existe';

  DROP TABLE ejemplo;
END;
```

Este procedimiento no fallará si la tabla no está: simplemente mostrará el mensaje "La tabla no existe" y continuará. Ahora bien, si el mismo handler se declara en un bloque más externo, y el error ocurre en un bloque más interno, ese



handler no tendría efecto. Por eso, es fundamental colocar los handlers en el lugar correcto.



Bibliografía

<https://dev.mysql.com/doc/refman/8.0/en/handler-scope.html>

<https://dev.mysql.com/doc/refman/8.0/en/signal.html#signal-condition-information-items>

https://www.tutorialspoint.com/mysql/mysql_signal_statement.htm